

MATH 345 Skeleton Notes

Unit 5: Optimization and training

Section 5.8: Applications and computation recap

© 2024–2026 Jacob Denson, Kaitlyn Phillipson, Sebastien Roch. Except where otherwise noted, this work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Application question. How do we read the short computations used in Unit 5?

This section checks whether you can connect a short calculation or code fragment to the mathematics in this unit. The goal is not to memorize Python. The goal is to identify the objects, check the shapes, and interpret the mathematical operation.

A short Unit 5 computation should be read in three steps.

1. Identify the mathematical objects.
2. Identify the update, matrix product, eigenvalue computation, or least-squares problem.
3. Interpret what the output says about optimization.

- **Code fragment:** `grad_f(x)`

First question to ask: What function is being minimized?

Typical interpretation: Gradient at the current point.

- **Code fragment:** `x - alpha * grad_f(x)`

First question to ask: Is the sign negative?

Typical interpretation: One gradient descent step.

- **Code fragment:** `losses.append(f(x))`

First question to ask: What is being recorded?

Typical interpretation: Loss history during an iteration.

- **Code fragment:** `np.linalg.eig(H)`
First question to ask: Is H a Hessian or another square matrix?
Typical interpretation: Eigenvalues and eigenvectors.
- **Code fragment:** `Q.T @ H @ Q`
First question to ask: Are the columns of Q orthonormal eigenvectors?
Typical interpretation: Hessian in principal-axis coordinates.
- **Code fragment:** `np.linalg.lstsq(A, b, rcond=None)[0]`
First question to ask: What are the columns of A ?
Typical interpretation: Least-squares coefficient vector.
- **Code fragment:** `r = b - A @ xhat`
First question to ask: What is being compared?
Typical interpretation: Residual vector.
- **Code fragment:** `A.T @ r`
First question to ask: Which Unit 4 condition is being checked?
Typical interpretation: Residual orthogonality.
- **Code fragment:** `np.column_stack(...)`
First question to ask: What features are being collected?
Typical interpretation: Design matrix construction.
- **Code fragment:** `np.outer(g, h)`
First question to ask: What are the shapes of g and h ?
Typical interpretation: Rank-one outer product.

Review activities

Activity: Reading a gradient descent loop

Assume `grad_f(x)` computes $\nabla f(x)$. Consider the code:

```
x = x0
for k in range(num_steps):
```

```
x = x - alpha * grad_f(x)
```

1. What mathematical update rule is represented?
2. Which quantity is the learning rate?
3. What shape must `grad_f(x)` have?
4. What would change if the minus sign were a plus sign?
5. Does this code prove that a minimum has been found?

Tags. [U5-L02, U5-L03 | C+T | Core]

The loop represents the update

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k).$$

The learning rate is `alpha`. The vector `grad_f(x)` must have the same shape as `x`, since the update subtracts one vector from another. If the minus sign were replaced by a plus sign, the update would move in the direction of steepest increase instead of steepest decrease. The loop does not prove that a minimum has been found. It only describes the repeated update; convergence depends on the function,

starting point, learning rate, and stopping rule.

Activity: Diagnosing learning rates

For $f(x) = x^2$, start at $x_0 = 4$ and use the update

$$x_{k+1} = x_k - \alpha f'(x_k).$$

The table shows the loss values $f(x_k)$ for three learning rates.

- $k: 0$
 - $\alpha = 0.05: 16.000$
 - $\alpha = 0.20: 16.000$
 - $\alpha = 1.05: 16.000$
- $k: 1$
 - $\alpha = 0.05: 12.960$
 - $\alpha = 0.20: 5.760$
 - $\alpha = 1.05: 19.360$
- $k: 2$
 - $\alpha = 0.05: 10.498$
 - $\alpha = 0.20: 2.074$
 - $\alpha = 1.05: 23.426$
- $k: 3$
 - $\alpha = 0.05: 8.503$
 - $\alpha = 0.20: 0.746$
 - $\alpha = 1.05: 28.345$

1. Which learning rate is making slow but steady progress?
2. Which learning rate is making faster useful progress?

3. Which learning rate appears unstable?
4. Does a decreasing loss table prove that the global minimum has been found?

Tags. [U5-L03 | C+T | Core]

The learning rate $\alpha = 0.05$ is making slow but steady progress. The learning rate $\alpha = 0.20$ is making faster useful progress. The learning rate $\alpha = 1.05$ appears unstable because the loss is increasing. A decreasing loss table does not prove that the global minimum has been found. It only shows what happened for the displayed iterates.

Activity: Reading Hessian eigenvalues

Suppose a critical point has Hessian matrix

```
H = np.array([[4.0, 0.0],
              [0.0, -1.0]])

evals, evecs = np.linalg.eig(H)
evals
```

Output:

```
array([ 4., -1.]
```

1. What matrix is being studied?
2. What do the signs of the eigenvalues say about the quadratic form $\mathbf{h}^T H \mathbf{h}$?
3. If H is the Hessian at a critical point, what does the second derivative test conclude?
4. Why is the phrase "at a critical point" important?

Tags. [U5-L04, U5-L05 | C+T | Core]

The matrix being studied is the Hessian matrix. Its eigenvalues have opposite signs, so the quadratic form $\mathbf{h}^T H \mathbf{h}$ takes both positive and negative values. If H is the Hessian at a critical point, the second derivative test classifies the critical point as a saddle point. The phrase "at a critical point" is important because the Hessian test classifies local behavior after the first-order term has disappeared.

Activity: Reading a least-squares residual check

Consider the code:

```
xhat = np.linalg.lstsq(A, b, rcond=None)[0]
r = b - A @ xhat
```

$A^T r$

1. What is $\hat{x}$?
2. What is r ?
3. What condition is checked by $A^T r$?
4. Does $A^T r$ being close to zero mean that r is close to zero?
5. Which earlier unit used this same condition?

Tags. [U5-L06, U4-L04 | C+T | Core]

The vector $\hat{x}$ is a least-squares coefficient vector. The vector r is the residual

$$r = b - A\hat{x}.$$

The expression $A^T r$ checks whether the residual is orthogonal to the columns of A . It can be close to zero even when r is not close to zero. Unit 4 used this same residual-orthogonality condition to derive the normal equations. Unit 5 obtains the same equations by setting the gradient of $\|Ax - b\|^2$ equal to zero.

Activity: Reading fixed-hidden-layer training

Let $\sigma(t) = \tanh(t)$. Consider the code:

```
A = np.column_stack([
    np.ones_like(t),
    np.tanh(t),
    np.tanh(t - 1),
    np.tanh(t + 1)
])

c = np.linalg.lstsq(A, y, rcond=None)[0]
yhat = A @ c
```

1. What are the columns of A ?
2. Which vector is being trained?
3. Why is this a least-squares problem?
4. Is the model linear as a function of t ?
5. Is the model linear as a function of c ?

Tags. [U5-L06, U5-L07 | C+M+T | Core]

The columns of A are the fixed feature vectors

$1, \quad \tanh(t), \quad \tanh(t - 1), \quad \tanh(t + 1),$
evaluated at the data inputs. The vector being trained is

$$\mathbf{c} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}.$$

This is a least-squares problem because the prediction vector is $A\mathbf{c}$, and training chooses $\mathbf{c}$ so that $A\mathbf{c}$ is close to $\mathbf{y}$. The model is not linear as a function of t , because it contains $\tanh(t)$, $\tanh(t - 1)$, and $\tanh(t + 1)$. It is linear as a function of $\mathbf{c}$, which is why least squares applies.

Activity: Reading a rank-one update

Suppose $\mathbf{g} \in \mathbb{R}^m$ and $\mathbf{h} \in \mathbb{R}^d$. Consider the code:

```
G = np.outer(g, h)
W_new = W - alpha * G
```

1. What is the shape of G ?
2. Why is `np.outer(g, h)` used here?
3. What mathematical update is represented by the second line?
4. Why is this connected to rank?

Tags. [U5-L07, U2-L03 | C+T | Core]

The matrix G has shape $m \times d$. The command `np.outer(g, h)` forms the matrix gh^T . This is the correct outer product when g and h are stored as one-dimensional NumPy arrays. The second line represents the update

$$W_{\text{new}} = W - \alpha gh^T.$$

The matrix gh^T has rank at most one, so the update changes W in one rank-one direction.

Linked notebook

Run Lab U5: Gradient descent and tiny training. Focus on four reading habits: identify the gradient descent update, diagnose a learning-rate table, recognize a least-squares call, and identify the fixed feature columns in the design matrix.

For a quick reference on Python loops, helper functions, entrywise functions, design matrices, eigenvalue and least-squares commands, transposes, and numerical checks such as "close to zero," see the programming appendix sections Small Python Patterns Used in the Labs, Elementwise Arithmetic Versus Linear Algebra, Constructing Common Arrays and Design Matrices, NumPy Linear Algebra Commands, and Numerical Checks and Roundoff.

Exam skill. Given a short Unit 5 code snippet, loss table, or matrix output, identify the mathematical operation and interpret the result as a descent step, Hessian/eigenvalue check, least-squares fit, residual-orthogonality

check, fixed-feature training problem, or rank-one update. *Tags.* [U5-L02, U5-L03, U5-L04, U5-L05, U5-L06, U5-L07 | C+M+T | Core]